- COL 1101

LECTURE 12

AUGUST 18, 2026

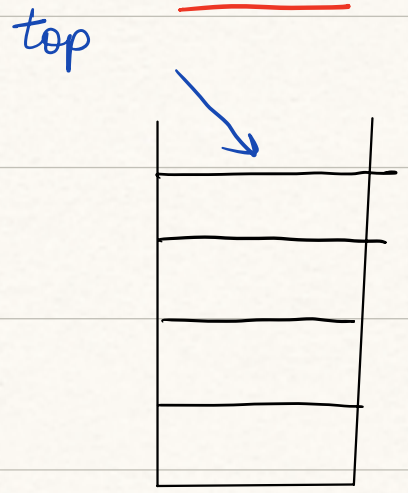
STACKS & QUEUE APPLICATIONS

+

INTRO TO FUNCTIONAL PROGRAMMING

Last Class: Stack and Queue Applications

STACK

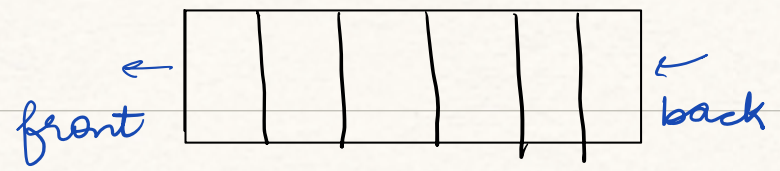


Last-in-First-Out
(LIFO)

Applications:

- Evaluating postfix expression
- Syntax validation

QUEUE



First-in-First-Out
(FIFO)

Applications

- Shortest path finding

MAXIMUM IN EVERY WINDOW

Problem: A stream of readings and a window of last k, sliding forward one step at a time. At every step, report the max inside the window.

MAXIMUM IN EVERY WINDOW

Problem: A stream of readings and a window of last k, sliding forward one step at a time. At every step, report the max inside the window.

Example: [4 2 5 1 6 3]

k = 3

5 5 6 6

MAXIMUM IN EVERY WINDOW

Problem: A stream of readings and a window of last k, sliding forward one step at a time. At every step, report the max inside the window.

Example: [4 2 5 1 6 3]

Brute-force approach; - Iterate over windows. ~ n

- In a window, scan all values to find the max ~ k operations

O(nk)

Can we do better?

MAXIMUM IN EVERY WINDOW

Key idea : When a bigger value arrives, the smaller values before are useless

Example

k = 4

#1 [5 3 1 _]

4

Safely discard these

#2: [6 5 4 _]

3

MAXIMUM IN EVERY WINDOW

Invariant: Maintain indices whose values are in decreasing order

MAXIMUM IN EVERY WINDOW

Invariant: Maintain indices whose values are in decreasing order

= P

Algorithm: $[a_0 \ldots a_n]$ ← remove element

remove element ← $[i_1\ i_2 \ldots i_j]$ ← i

access element P for current window

For each new window ← add element

- remove the earliest index if it fell out of window
- remove all indices with values less than current element & append new element to P
- max is the leftmost element

Double-ended queue (dequeue) → add left, add right, remove left, remove right

CODE

```
from collections import deque

def max_in_windows(a, k):
    dq = deque()
    out = []
    for i, x in enumerate(a):
        if dq and dq[0] <= i - k:
            dq.popleft()
        while dq and a[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if i >= k - 1:
            out.append(a[dq[0]])
    return out
```

O(1)

O(1)

O(1)

TIME COMPLEXITY ?

O(n)

for each element

→ added & removed

to the deque

exactly one

→

loop over : O(n)

n operation

STACK AND QUEUE APPLICATIONS

Stacks

- Evaluating postfix expression
- Syntax validation
- Undo / Redo
- Recursion
-

Queues

- Scheduling
- Buffer (e.g. video buffer on Youtube)
- Shortest path finding

A

-

Stack And Queue Applications

Stacks

- Evaluating postfix expression
- Syntax validation
- Undo / Redo
- Recursion
-

Queues

- Scheduling
- Buffer (e.g. video buffer in Youtube)
- Shortest path finding

Don't need to remember!

A

Given a problem - analyze the access pattern and ordering rule the problem requires

-

LET'S TAKE A STEP BACK

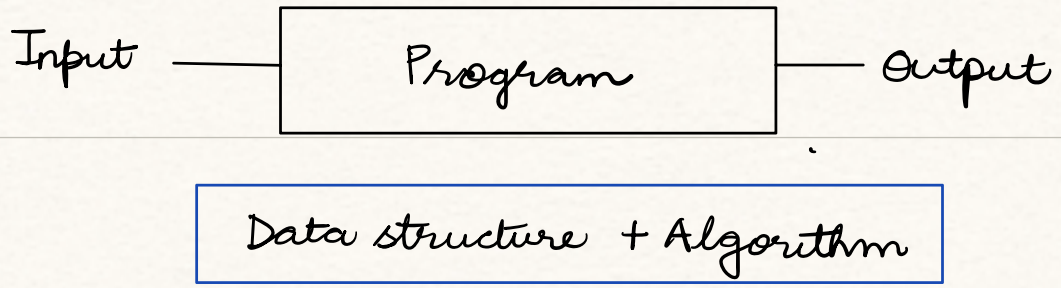


LET'S TAKE A STEP BACK



Data structure + Algorithm

LET'S TAKE A STEP BACK



Object-oriented Programming welds the data (attributes) to the algorithms that act on it (methods)

class

Example : LinkedStack :

- l = [ ], top, length

operation { push, pop, size

NEXT FEW CLASSES: FUNCTIONAL PROGRAMMING

An alternate programming paradigm that instead of fusing data & algorithms (which OOP does), pulls them apart.

- data is plain and never changes - immutable

NEXT FEW CLASSES: FUNCTIONAL PROGRAMMING

An alternate programming paradigm that instead of fusing data & algorithms (which OOP does), pulls them apart.

- data is plain and never changes - immutable
- algorithms are standalone functions with no hidden state - pure functions

NEXT FEW CLASSES: FUNCTIONAL PROGRAMMING

An alternate programming paradigm that instead of fusing data & algorithms (which OOP does), pulls them apart.

- data is plain and never changes - immutable
- algorithms are standalone functions with no hidden state - pure functions
- build programs by composing functions, not mutating objects.

NEXT FEW CLASSES: FUNCTIONAL PROGRAMMING

An alternate programming paradigm that instead of fusing data & algorithms (which OOP does), pulls them apart.

- data is plain and never changes - immutable
- algorithms are standalone functions with no hidden state - pure functions
- build programs by composing functions, not mutating objects.

DISCLAIMER: Not strictly better, but different paradigm

WHERE OOP DIVERGES FROM FUNCTIONAL PROGRAMMING.

```
s = LinkedStack()
s.push('a')
s.push('b')
```

backup = s → reference aliasing

→ s.deep_copy()

→ s.pop()

```
backup.size()
```

What is the size of backup?

↳ 1

O(n)

→ methods can mutate the data

PROBLEM 2 : BLOATED CLASS

Scenario : You have a standard Queue class (enqueue, dequeue). You are asked to find the highest number inside the queue.

↳ write an external method

copy ↓

- ↳ unpack the data (dequeue)
- ↳ pack the data (enqueue)

↳ write an internal method

↳ Enqueue, dequeue, findMax, findMin, findMedian

PROBLEM 2 : BLOATED CLASS

Scenario : You have a standard Queue class (enqueue, dequeue). You are asked to find the highest number inside the queue.

Bad Choice You add findMax() inside the class.

PROBLEM 2 : BLOATED CLASS

Scenario : You have a standard Queue class (enqueue, dequeue). You are asked to find the highest number inside the queue.

Bad Choice 1: You add findMax() inside the class. Class can become bloated.

Bad choice 2: You write code outside the class. Because data is hidden*, you have to unpack (dequeue) & pack (enqueue) the data.

PROBLEM 2 : BLOATED CLASS

Scenario : You have a standard Queue class (enqueue, dequeue). You are asked to find the highest number inside the queue.

Bad Choice 1: You add findMax() inside the class. Class can become bloated.

Bad choice 2: You write code outside the class.

Issue: Data & operations are coupled together

THE FP SOLUTION: DECOUPLE DATA & OPERATIONS

Function 1

```
def _append4(l)
    l.append(4)
```

Function #2

```
def _append4(l)
    l1 = l + [4]
    return l1
```

what is the ouput of _append4(l) x 3 ?

Function 1: [1, 2, 3, 4, 4, 4]

l = [1, 2, 3]

Function 2: [1, 2, 3, 4]

→ Data is immutable + You write pure functions

THE FP SOLUTION: DECOUPLE DATA & OPERATIONS

Function 1

```
def _append4(l):
    l.append(4)
```

Function #2

```
def _append4(l):
    l1 = l + [4]
    return l1
```

what is the ouput of _append(4) x 3 ?

.

THE FP SOLUTION: DECOUPLE DATA & OPERATIONS

Function 1

```
def append4(l):
    l.append(4)
```

Function #2

```
def append4(l):
    l1 = l + [4]
    return l1
```

- Data is immutable
- You write pure functions

PURE FUNCTION

Definition : A function is pure when :

1). its output depends only on its inputs ✓

2). it produces no side effects. Calling it changes nothing observable outside the function.

Example :

```
def add (x, y):
    return x + y
```

Exercise

```
def square(x):      return x * x
```

Pure function

Exercise

```
def square(x):        return x * x
def greet(name):      print("hi", name)
```

→ Not pure function

Not a pure

EXERCISE

```
def square(x):      return x * x
def greet(name):    print("hi", name)
def roll():         return random.randint(1, 6)
```

roll() + roll() = 2 x roll() X

Exercise

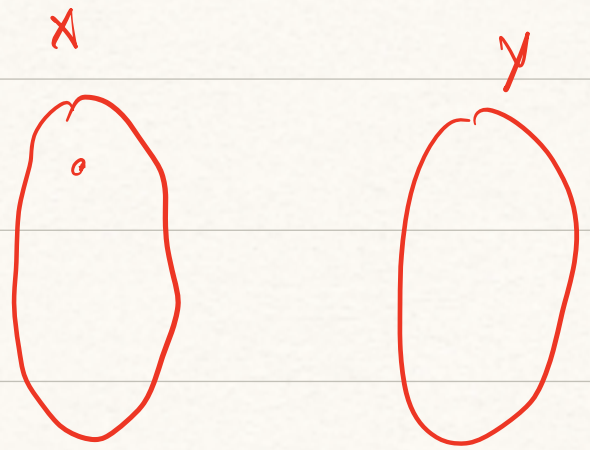
```
def square(x):      return x * x
def greet(name):    print("hi", name)
def roll():         return random.randint(1, 6)
def now():          return time.time()
```

→ Not a pure function

now ( )

Clause 1 : function is a pure mathematical function

ϕ



EXERCISE

```
def square(x):      return x * x
def greet(name):    print("hi", name)
def roll():         return random.randint(1, 6)
def now():          return time.time()
def tax(amount):    return amount * RATE      #
```

↳ Not a pure fun

amount x 100

tax(3)

amount x math.pi

EXERCISE

```
def square(x):      return x * x
def greet(name):    print("hi", name)
def roll():         return random.randint(1, 6)
def now():          return time.time()
def tax(amount):    return amount * RATE      # RATE = 0.07
def push(stack, x): stack.append(x)
```

✓

✗ side effect

✗ clause 1

✗ clause 1

✗ clause 1

✗ side effect

clause 2 is not satisfied

WHAT ARE THE BENEFITS?

1. Simpler testing and debugging

```
assert square(5) == 25
```

- pop()

WHAT ARE THE BENEFITS?

referential transparency

1. Simpler testing and debugging
2. Cacheable (Memoization)

```
from functools import lru_cache

@lru_cache(maxsize=None)
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

WHAT ARE THE BENEFITS?

1. Simpler testing and debugging
2. Cacheable
3. Concurrency is easier

Did we write pure data structures?

Recall stack push operation

```
def push(self, x):
    self.top = Node(x, self.top)
```

WHAT WOULD A PURE "PUSH" LOOK LIKE?

Implement push function on a stack

push (S, x)

S' → which has also the element added

returns S'

↳ This "may be" costly

↓

It can be made cheaper in some context

WHAT WOULD A PURE "PUSH" LOOK LIKE?